- [ ] Key-based access tested (multi-session, regeneration)
- [ ] Comparison with old 23-node pipeline

---

19. Dashboard Production Readiness

19.1 Strategic Positioning

The positioning is fixed: PARWA is a headcount replacement platform.

This is the bold CFO-facing version, not the soft CSM-facing version. Every dashboard element, every metric, every notification must reinforce this positioning. The buyer is the CFO/VP Support/COO — they are buying PARWA to eliminate headcount cost, not to "make their team more productive."

The moment PARWA's dashboard starts showing "your team is more productive" framing, we lose the differentiation against every other CX tool on the market. The competitive moat is the **headcount cost elimination** narrative, backed by the ROI Calculator showing real dollars saved.

The softer "CSM superpowers" positioning is reserved for **internal champions** during the sales discovery process — once the CFO has approved the budget based on headcount savings, the internal champion (often a CS Ops lead) gets the "this makes your team 5x more effective" pitch to drive adoption. But the dashboard itself never softens — it always shows replacement dollars.

19.2 Dashboard Removals — Delete These Features

The current dashboard has features that contradict the headcount-replacement positioning or that expose internal architecture to end users. These must be removed before production.

19.2.1 Remove: Agent Builder (`/dashboard/agents/new`)

Current state: Dead UI page with no backend wiring. Shows a fake "create new agent" form that does nothing.

Why remove entirely:
- The architecture is **one agent per variant** with dynamic restrictions applied at runtime. There is no concept of "creating an agent" — the agent IS the variant.
- Exposing an agent builder implies PARWA is a "build your own AI agent" platform (AgentGPT, AutoGen, etc.) — that's a different market with different buyers and a worse unit economics.
- The agent builder creates the false impression that PARWA requires configuration work from the customer. It does not — PARWA works out of the box with the 6-step onboarding.

Action: Delete the page, remove from navigation, remove from any internal links. Update sidebar nav to show only "Variants" (which already exists and works).

Files to delete:
- `src/app/dashboard/agents/new/page.tsx`
- `src/app/dashboard/agents/page.tsx` (listing page — also unused)
- `src/components/dashboard/agent-builder/*` (entire folder if exists)
- Any Zustand store or hook referencing `agentBuilder`

19.2.2 Remove: Pipeline Visualization Page

Current state: Mock page showing the 8 PARWA nodes with fake status indicators.

Why remove:
- The 8-node PARWA pipeline is **internal architecture**. Showing it to end users is like Stripe showing their internal payment routing diagram to merchants. Customers don't care HOW the sausage is made — they care that tickets get resolved.
- Showing the pipeline invites the question "can I configure individual nodes?" — and the answer is no, which frustrates users.
- The pipeline visualization creates a false perception that PARWA requires monitoring. It does not — Jarvis monitors itself and surfaces only actionable issues via the Notification Center.

What replaces it: A simple **System Health** indicator in the dashboard header (green/yellow/red dot) that reflects overall platform status. If green, no user action needed. If yellow/red, click to see Jarvis's diagnosis. No 8-node diagram, no per-node status, no technical jargon.

Action: Delete the pipeline page, replace with a single status chip in the top nav.

Files to delete:
- `src/app/dashboard/pipeline/page.tsx`
- Any pipeline visualization component folder
- Update sidebar nav to remove "Pipeline" entry

19.2.3 Remove: AI Wiki Viewer (Backend-Only Asset)

Current state: Planned but not yet built. Some wireframes show a "View AI Wiki" page in the dashboard.

Why NOT to build it:
- The AI Wiki is **food for the agents, not content for humans**. It's structured data (ticket patterns, admin behaviors, company knowledge) that the Reasoning Engine retrieves during ticket resolution.

- A human reading the AI Wiki would see something like: "PATTERN-4823: When user reports 'login failed' + tenant=AcmeCorp + time=09:00-11:00 EST → 87% probability of Okta sync issue. Action: check Okta connector, escalate to admin if persists." This is unreadable noise for a human but gold for an LLM.
- Showing the AI Wiki implies the customer needs to "review" or "approve" what the AI knows. They do not. The customer approved everything during the 6-step onboarding; thereafter PARWA manages the Wiki autonomously.

Exception — what the customer CAN see:
- A **read-only "Knowledge Sources" panel** showing what documents they've uploaded (Section C — Company Knowledge). They can add/remove documents here. This is content management, not Wiki viewing.
- They do NOT see Section A (Ticket Patterns — PARWA writes) or Section B (Admin Behavior — Jarvis writes).

Action: Do NOT build an AI Wiki viewer. Build a "Knowledge Sources" file manager only (for Section C content the admin uploads). Sections A and B remain backend-only.

19.2.4 Remove: Shadow Mode Page

Current state: Silently 404s. No backend route.

Why remove: Shadow mode (PARWA resolving tickets in parallel with humans for benchmarking) is a **sales/demo feature**, not a production feature. It belongs in a separate internal tool, not the customer dashboard.

Action: Delete the nav entry. If shadow benchmarking is needed for a specific deal, build it as a one-off internal script, not a dashboard feature.

19.2.5 Remove: All Mock/Hardcoded Pages

Current state: Billing page (hardcoded numbers), Notifications page (mock array), Profile page (stale localStorage).

Why remove: These are broken. Either wire them to real backend data or remove them. There is no middle ground for production.

Action: See Section 19.4 (Wiring Tasks) for what each must connect to.

19.3 Dashboard Additions — New Features to Build

These are the four Jarvis-powered features plus the ROI Calculator that close the "replace the CSM" pitch. Without these, the headcount-replacement story has gaps. With these, the pitch is airtight.

19.3.1 ADD: SLA Tracking (Jarvis Pre-Breach Alerts)

What it does: Jarvis actively monitors every ticket's response-time SLA against the contracted SLA tier. When a ticket is approaching breach (configurable threshold: default 80% of SLA window elapsed), Jarvis fires a notification to the admin BEFORE the breach happens — not after.

Why this matters: CSMs currently maintain SLA compliance by manually watching dashboards. PARWA replaces that manual monitoring with continuous Jarvis surveillance. This is one of the 8 PARWA-replaceable CSM duties.

Data flow:
1. Each ticket ingested via Node 1 (Ingest+Classify) is tagged with the customer's SLA tier (e.g., "Critical: 1hr response, 4hr resolution")
2. Jarvis SENSE continuously polls ticket state every 30 seconds
3. Jarvis EVALUATE computes remaining SLA window per ticket
4. At 80% elapsed → Jarvis NOTIFY sends "SLA at risk" alert to admin (and assigned human if any)
5. At 100% elapsed → Jarvis NOTIFY sends "SLA breached" alert with auto-generated postmortem

Dashboard UI:
- **SLA Dashboard page** with three sections:
  - **At-risk tickets** (red, 80-100% elapsed) — sortable by time remaining
  - **Breached tickets** (last 7 days, with root-cause analysis from Jarvis)
  - **SLA compliance trend** (line chart, last 30 days, per SLA tier)
- Per-ticket detail: SLA clock, time elapsed, time remaining, who Jarvis alerted, when

Backend requirements:
- New table: `sla_policies` (per tenant, per customer tier)
- New table: `sla_events` (every SLA at-risk + breach event, with Jarvis analysis)
- New Jarvis polling job (every 30s per tenant)
- New API routes: `GET /api/v1/sla/at-risk`, `GET /api/v1/sla/breaches`, `GET /api/v1/sla/compliance`

Positioning in pitch: *"Your CSM job posting says 'maintain SLA compliance.' That's a $95K/year human watching a clock. PARWA watches the clock in real time and alerts you before the breach — not after."*

19.3.2 ADD: Customer Health Scores (Jarvis-Computed)

What it does: Jarvis computes a real-time health score (0-100) for every customer/account in the system, based on a weighted formula across multiple signals. CSMs currently do this manually in spreadsheets.

Why this matters: Customer health scoring is the second-most-cited CSM duty in job postings. Without this feature, the "replace the CSM" pitch has a visible hole. With it, PARWA covers the entire CS operational stack.

Health score formula (default, admin-configurable per tenant):
- **Ticket volume trend** (25%): decreasing = healthy, increasing = unhealthy
- **Sentiment trend** (20%): aggregated sentiment across all customer tickets (Node 6 Quality+Format produces sentiment score per ticket)
- **Resolution rate** (20%): % of tickets resolved by PARWA without human escalation
- **Time-to-resolution** (15%): trend vs. customer's historical baseline
- **Reopen rate** (10%): % of tickets reopened by customer after "resolved"
- **Product usage** (10%): pulled from customer's product analytics (via UCB integration to Segment/Amplitude/Mixpanel)

Data flow:
1. Jarvis SENSE ingests all 6 signals continuously (or nightly for usage data)
2. Jarvis EVALUATE computes weighted score per account, stores in `customer_health` table
3. Jarvis EVALUATE detects score drops >10 points in 7 days → flags for proactive outreach (see 19.3.4)
4. Dashboard queries `customer_health` table for display

Dashboard UI:
- **Customer Health page** with:
  - **Health matrix**: all accounts on a 0-100 color-coded grid (red <40, yellow 40-70, green >70)
  - **Trend chart**: per-account 90-day health trend
  - **Signal breakdown**: which signals are dragging the score down
  - **Drill-down**: click any account → see ticket history, sentiment trend, usage trend, last 5 Jarvis evaluations

Backend requirements:
- New table: `customer_health` (account_id, score, signal_breakdown_json, updated_at)
- New table: `health_signals` (per-signal raw values per account per day)
- New Jarvis EVALUATE job (hourly per tenant)
- New API routes: `GET /api/v1/health/accounts`, `GET /api/v1/health/accounts/:id`, `GET /api/v1/health/accounts/:id/trend`
- UCB connectors for Segment, Amplitude, Mixpanel (for usage signal)

Positioning in pitch: *"Your CSM maintains a customer health spreadsheet. PARWA computes it continuously from 6 live signals — and tells you which signal is dragging the score down. No spreadsheet updates, no quarterly review panic."*

19.3.3 ADD: QBR / Report Generator (One-Click)

What it does: A single button — "Generate Quarterly Report" — that compiles everything Jarvis knows about a customer into a polished PDF executive summary. CSMs spend days on these; PARWA generates in 5 seconds.

Why this matters: QBR generation is one of the most-cited CSM time-sinks in job postings. It's also a high-leverage moment — the QBR is often where renewals are won or lost. Automating it removes busywork AND improves quality (Jarvis sees more than any human CSM could).

Report contents (auto-compiled):
1. **Executive summary** (1 paragraph, LLM-generated from ticket + health data)
2. **Ticket metrics** (volume, resolution rate, avg time-to-resolve, top 5 categories)
3. **SLA performance** (compliance %, breaches with root cause)
4. **Customer health trend** (chart + commentary)
5. **Sentiment trend** (chart + notable moments)
6. **Top resolved issues** (5 most impactful resolutions, with PARWA credit)
7. **Open risks** (Jarvis-flagged concerns going into next quarter)
8. **Recommendations** (LLM-generated next-quarter action items)
9. **ROI summary** (dollars saved this quarter vs. PARWA cost — pulled from ROI Calculator)

Data flow:
1. Admin clicks "Generate QBR" button on any account page
2. Backend pulls all relevant data from `tickets`, `sla_events`, `customer_health`, `roi_events` tables for the selected quarter
3. LLM generates executive summary + recommendations (single LLM call, ~2k tokens)
4. ReportLab renders PDF with charts (matplotlib PNGs embedded)
5. PDF saved to tenant's S3 bucket, signed URL returned to dashboard
6. User downloads PDF or schedules auto-send to customer

Dashboard UI:
- "Generate QBR" button on every account page
- QBR history list (last 12 reports per account)
- Optional: schedule auto-generation (e.g., generate first day of each quarter, email to admin)

Backend requirements:
- New endpoint: `POST /api/v1/reports/qbr` (input: account_id, quarter)
- New endpoint: `GET /api/v1/reports/qbr/:id/download`
- LLM call: GPT-4-class model, structured prompt with all data injected

- PDF generation via ReportLab (existing pattern from PDF skill)
- Charts via matplotlib (existing pattern)
- S3 storage for generated PDFs

Positioning in pitch: *"Your CSM spends 3 days per quarter per account building QBR decks. PARWA generates them in 5 seconds with more data than any human could compile. Your CSM reviews, edits, and presents — the busywork is gone."*

19.3.4 ADD: Proactive Outreach Suggestions (Jarvis-Initiated)

What it does: Jarvis continuously analyzes every account for "reach out" signals and surfaces specific, actionable suggestions to the admin: "Customer X hasn't opened a ticket in 60 days but their product usage dropped 40% — reach out." CSMs do this from gut feel today; PARWA does it from data.

Why this matters: This is the proactive-retention muscle that justifies the CSM salary. Without it, PARWA is reactive (resolves tickets well) but not proactive (prevents churn). Adding this closes the loop and makes the CSM replacement airtight.

Signal categories Jarvis watches for:
- **Usage drop**: product usage down >30% in 30 days (via UCB → Segment/Amplitude)
- **Silence anomaly**: no tickets in 60+ days when historical avg is 5+/month (could mean churned silently or gave up)
- **Sentiment decline**: rolling 30-day sentiment down >15 points vs. previous 30 days
- **Ticket spike**: 2x normal ticket volume in 7 days (frustration building)
- **Reopen spike**: reopen rate >20% in last 14 days (resolutions not sticking)
- **SLA breach cluster**: 2+ SLA breaches in 30 days for same account
- **Admin policy change**: admin updated a policy that affects this account (Jarvis detects via Policy Change Detection — see Section 10)
- **Billing event**: failed payment, downgrade, or renewal approaching (via UCB → Stripe/billing system)

How Jarvis surfaces suggestions:
- New dashboard widget: **Outreach Suggestions** (top of main dashboard, can't miss it)
- Each suggestion card shows: account name, signal type, severity (low/med/high), suggested action (LLM-generated email draft included), "Send" or "Dismiss" buttons
- Admin can: send the drafted email as-is, edit then send, dismiss (with reason — feeds back into Jarvis learning), or assign to a human teammate
- All outreach actions logged in `outreach_log` table for audit

Data flow:
1. Jarvis EVALUATE runs nightly per tenant, scans every account for all 8 signal categories
2. New signals → inserted into `outreach_suggestions` table
3. Jarvis generates suggested email draft via LLM (single call per suggestion, ~500 tokens)

4. Dashboard polls `outreach_suggestions` table on page load + every 60s
5. Admin action updates suggestion status → feeds back into AI Wiki Section B (admin behavior)

Dashboard UI:
- **Outreach Queue** page (full list, filterable by severity, signal type, account)
- **Outreach widget** on main dashboard (top 3 highest-severity, click to expand)
- Per-suggestion card: account, signal, suggested action, draft email, action buttons
- **History view**: all past suggestions with outcome (sent, dismissed, converted to ticket, etc.)

Backend requirements:
- New table: `outreach_suggestions` (account_id, signal_type, severity, draft_email, status, created_at, resolved_at)
- New table: `outreach_log` (every send/dismiss/assign action)
- New Jarvis EVALUATE job (nightly per tenant)
- New API routes: `GET /api/v1/outreach/suggestions`, `POST /api/v1/outreach/suggestions/:id/send`, `POST /api/v1/outreach/suggestions/:id/dismiss`

Positioning in pitch: *"Your CSM's gut says 'I should check in on Customer X.' PARWA's data says 'Customer X's usage dropped 40%, no tickets in 60 days, last sentiment score was 45/100 — here's a draft email, want me to send it?' That's not a CSM replacement. That's a CSM upgrade."*

19.3.5 ADD: ROI Calculator (Wire Already-Coded UI to Live Data)

Current state: UI is already coded. Currently shows static/demo numbers. Needs to be wired to live backend data so the numbers reflect the actual tenant's real savings.

What it does: Every time the admin logs in, the dashboard header shows a live counter: "PARWA has saved you $X this month by replacing Y hours of CSM/CSR work." Click to expand → full ROI breakdown page.

Why this matters: This is the single highest-converting sales asset in the entire platform. Every login reminds the buyer why they're paying for PARWA. It makes the value visceral — not abstract, not promised, but realized and counted.

What "savings" are computed (per tenant, per month):

Cost Category	Computation	Source Data
Tier 1 Support hours saved	(Tickets auto-resolved by PARWA) × (avg 12 min per ticket × $23/hr loaded Tier 1 cost)	`tickets` table where `resolved_by='parwa'` and `complexity='tier1'`
CSR hours saved	(Tickets auto-resolved) × (avg 8 min × $20/hr)	`tickets` table where `resolved_by='parwa'` and `complexity='basic'`

CSM hours saved	(Outreach suggestions auto-generated) × (avg 20 min × $45/hr) + (QBRs auto-generated × 24 hrs × $45/hr)	`outreach_suggestions` + `reports` tables
SLA breach cost avoided	(Breaches prevented by Jarvis pre-alert) × (avg breach penalty $500)	`sla_events` where `prevented=true`
Onboarding hours saved	(Customers onboarded via 6-step flow) × (avg 8 hrs × $45/hr)	`onboarding_events` table

Total monthly savings = sum of all 5 categories, minus PARWA's monthly cost (transparently shown).

Dashboard UI:
- **Live counter in top nav** (next to notifications bell): "Saved $X this month" — updates on every page load
- **ROI page** (`/dashboard/roi`):
  - Big number: total saved YTD
  - Breakdown chart: 5 cost categories as stacked bar chart, monthly
  - Trend line: savings over last 12 months
  - Comparison: "PARWA cost: $X/mo. Savings: $Y/mo. Net ROI: Zx"
  - "Share with my CFO" button → generates PDF executive summary of ROI

Backend requirements:
- New table: `roi_events` (every savings-eligible event logged with $ value)
- New materialized view: `roi_monthly` (rolled-up monthly savings per tenant)
- New API routes: `GET /api/v1/roi/summary`, `GET /api/v1/roi/breakdown`, `GET /api/v1/roi/trend`, `POST /api/v1/roi/cfo-report`
- Event hooks: every ticket resolution, every outreach suggestion, every QBR generation, every SLA prevention → insert row into `roi_events`

Action items (since UI is coded):
1. Identify the existing ROI Calculator component in the dashboard
2. Replace static/demo data with API calls to `/api/v1/roi/*`
3. Implement the 5 backend event hooks to populate `roi_events`
4. Build the materialized view for monthly rollup
5. Test with real tenant data (must show non-zero savings within first 24 hours of tenant using PARWA)

Positioning in pitch: *"Every time your CFO logs in, the first thing they see is how much money PARWA saved this month. Not 'tickets resolved' — actual dollars. That's the conversation you want to have with your CFO every month, not the 'how do we justify the AI tool spend' conversation."*

19.4 Dashboard Wiring Tasks — Fix the Broken Pages

Beyond the additions above, the following existing pages must be wired to real backend data before production. These were identified in the dashboard audit.

19.4.1 Tickets Page → Wire to `/api/v1/tickets/*`

Current: Uses localStorage only. Tickets created in dashboard never reach backend.

Required:
- Replace localStorage reads with `GET /api/v1/tickets` (with pagination, filtering)
- Replace localStorage writes with `POST /api/v1/tickets` (create), `PATCH /api/v1/tickets/:id` (update)
- Real-time updates via Socket.io (already in stack) — listen for `ticket:created`, `ticket:resolved`, `ticket:escalated` events
- Show ticket count badge in sidebar with live updates

19.4.2 Billing Page → Wire to `/api/billing/*`

Current: All hardcoded numbers.

Required:
- Pull current plan from `GET /api/billing/subscription`
- Pull usage this cycle from `GET /api/billing/usage`
- Pull invoice history from `GET /api/billing/invoices`
- Upgrade/downgrade flow via `POST /api/billing/subscription/change`
- Paddle integration already exists in backend — ensure webhook events update dashboard in real-time

19.4.3 Notifications Page → Wire to `/api/v1/notifications`

Current: Mock array of fake notifications.

Required:
- Pull from `GET /api/v1/notifications` (with filter for unread)
- Mark as read via `PATCH /api/v1/notifications/:id/read`
- Real-time push via Socket.io — `notification:new` event
- Notification types: SLA at-risk, SLA breached, outreach suggestion, QBR ready, policy change detected, ROI milestone reached

19.4.4 Profile Page → Wire to `/api/v1/auth/me`

Current: Reads stale localStorage.

Required:
- On page load, fetch fresh profile from `GET /api/v1/auth/me`

- Allow update via `PATCH /api/v1/auth/me` (name, avatar, preferences)
- Show last login, sessions list, API key regeneration (with admin-only permission check)
- 2FA setup flow if not yet enabled

19.4.5 Integrations Page → Wire to UCB Connector Status

Current: Shows static list of integrations with no live status.

Required:
- Pull connected integrations from `GET /api/v1/integrations`
- Per-integration health status (last sync time, errors, credentials valid)
- Add/remove integration flow (OAuth for SaaS, API key for others)
- Test integration button (sends a ping, verifies 200 response)

19.5 Dashboard Cleanup — Delete Dead Code

~30 dead code files identified in audit. These must be deleted before production to reduce maintenance surface and avoid confusing future developers.

Cleanup process:
1. Run dead code analysis: `npx ts-prune` to find unused exports
2. Cross-reference with file dependency tree (Next.js build output)
3. Manually verify each flagged file is truly unused (grep for imports)
4. Delete in batches: components → hooks → stores → utils → types
5. Run full test suite after each batch
6. Update sidebar nav, route definitions, and any dynamic imports

Expected outcome: ~30 files deleted, bundle size reduced ~15-20%, build time reduced ~30%.

19.6 Final Dashboard Information Architecture

After all removals and additions, the production dashboard sidebar should be:

```
DASHBOARD
├── Overview (main dashboard with ROI counter, outreach widget, SLA widget)
├── Tickets (live ticket queue, real-time)
├── Customer Health (health score matrix + trends)
├── SLA Tracking (at-risk, breaches, compliance trend)
├── Outreach Suggestions (Jarvis-proactive queue)
├── Reports (QBR generator, ROI report, history)

VARIANT
```

```
├── Variants (list, create, manage)
├── Knowledge Sources (Section C file manager only)

INTEGRATIONS
├── Connectors (UCB status, add/remove)
├── Notifications (Jarvis alerts, real-time)

ACCOUNT
├── Billing (real subscription, usage, invoices)
├── Profile (fresh from /auth/me)
├── Settings (tenant preferences, SLA policy config)
├── API Keys (admin-only, generate/revoke)
```

What is NOT in the sidebar:
- ❌ Agent Builder (removed — see 19.2.1)
- ❌ Pipeline visualization (removed — see 19.2.2)
- ❌ AI Wiki viewer (not built — see 19.2.3)
- ❌ Shadow Mode (removed — see 19.2.4)

19.7 Dashboard Production Readiness Checklist

Before declaring the dashboard production-ready, every item below must be verified:

Removals complete:
- [ ] Agent Builder page deleted, no remaining references in codebase
- [ ] Pipeline visualization page deleted, replaced with status chip in top nav
- [ ] AI Wiki viewer NOT built (only Knowledge Sources file manager built)
- [ ] Shadow Mode nav entry removed
- [ ] All mock/hardcoded data removed from Billing, Notifications, Profile

Additions complete:
- [ ] SLA Tracking page live, wired to Jarvis SENSE/EVALUATE/NOTIFY
- [ ] Customer Health page live, showing real scores from `customer_health` table
- [ ] QBR Generator page live, producing downloadable PDFs
- [ ] Outreach Suggestions page live, Jarvis generating nightly suggestions
- [ ] ROI Calculator wired to live `roi_events` data (counter in top nav showing real $)
- [ ] ROI "Share with CFO" PDF report generator working

Wiring complete:
- [ ] Tickets page reads/writes via `/api/v1/tickets/*`, real-time via Socket.io
- [ ] Billing page reads from `/api/billing/*`, Paddle webhooks updating in real-time
- [ ] Notifications page reads from `/api/v1/notifications`, real-time push working
- [ ] Profile page fetches fresh from `/api/v1/auth/me` on every load

- [ ] Integrations page shows live UCB connector status with test button

Cleanup complete:
- [ ] `npx ts-prune` run, all unused exports removed
- [ ] ~30 dead files deleted (verify with grep before deletion)
- [ ] Bundle size reduced (measure before/after)
- [ ] Build time reduced (measure before/after)
- [ ] Full E2E test suite passes on cleaned codebase

Positioning verification:
- [ ] No "make your team more productive" language anywhere in dashboard
- [ ] ROI counter visible on every page (top nav)
- [ ] Every dashboard widget ties back to a dollar savings or a headcount cost
- [ ] Headcount replacement language consistent across dashboard, onboarding, and docs

---

Appendix A: Node-by-Node LLM Call Summary (PARWA)